

Diseño y Análisis de Algoritmos

Pauta Taller 1: Invariante de Loop

Prof. Felipe Ruiz, Ayud. Ferran Rojas Andreu

Esta pauta contiene soluciones de referencia para los cuatro algoritmos propuestos. Cada solución incluye: pseudocódigo CLRS, invariante formal, prueba de las tres propiedades, análisis de complejidad, edge cases, variantes del invariante y tips para la presentación.

1. Insertion Sort

Pseudocódigo

```

1  procedure INSERTION-SORT( $A$ )
2    for  $j = 2$  to  $A.length$ 
3       $key = A[j]$                                 ▷ Insert  $A[j]$  into sorted sequence  $A[1..j-1]$ 
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6         $A[i + 1] = A[i]$ 
7         $i = i - 1$ 
8       $A[i + 1] = key$ 

```

Invariante de Loop

Para cada iteración del loop exterior con índice j ($2 \leq j \leq n + 1$), sea $I(j)$:

$A[1..j-1]$ es una permutación de $A_{\text{orig}}[1..j-1]$ y $A[1] \leq A[2] \leq \dots \leq A[j-1]$.

Equivalentemente: el subarreglo $A[1..j-1]$ está ordenado y contiene exactamente los mismos elementos que las primeras $j-1$ posiciones del arreglo original.

Prueba de las Tres Propiedades

Inicialización ($j = 2$)

Antes de la primera iteración, $j = 2$. El subarreglo $A[1..1]$ consiste en un único elemento $A[1]$. Un arreglo de un elemento está ordenado trivialmente y es permutación de sí mismo. Por tanto $I(2)$ se cumple.

Mantenimiento

Hipótesis inductiva: $I(j)$ es cierto al inicio de la iteración j ($2 \leq j \leq n$). El loop interior (**While**) desplaza hacia la derecha todos los elementos $A[i] > key$ en el subarreglo ordenado $A[1..j-1]$. Como $A[1..j-1]$ está ordenado (por HI), los elementos mayores que key forman un sufijo contiguo. El loop termina cuando $i = 0$ o $A[i] \leq key$, momento en que la posición correcta para key es $i + 1$. Al asignar $A[i + 1] = key$, el subarreglo $A[1..j]$ queda ordenado y contiene los elementos de $A_{\text{orig}}[1..j]$ (se agregó $A[j]$ original como key , y los demás solo se desplazaron). Por tanto $I(j + 1)$ se cumple.

Terminación

El loop exterior termina cuando $j = n + 1$. En ese momento $I(n + 1)$ afirma que $A[1..n]$ está ordenado y es permutación de $A_{\text{orig}}[1..n]$. Esto es exactamente la postcondición: el arreglo completo está ordenado.

Complejidad

- **Peor caso:** $\Theta(n^2)$. Ocurre cuando el arreglo está en orden estrictamente decreciente. El loop interior ejecuta $j - 1$ veces para cada j , total $\sum_{j=2}^n (j - 1) = n(n - 1)/2$.
- **Mejor caso:** $\Theta(n)$. Ocurre cuando el arreglo ya está ordenado. La condición $A[i] > \text{key}$ es falsa inmediatamente, el loop interior no ejecuta. Solo $n - 1$ comparaciones.
- **Caso promedio:** $\Theta(n^2)$. En una permutación aleatoria, en promedio key se inserta a la mitad del subarreglo ordenado, $\sim j/2$ iteraciones interiores.
- **Espacial:** $\Theta(1)$ extra (algoritmo *in-place*).

Edge Cases

- $n = 0$ o $n = 1$: loop exterior no ejecuta (j va de 2 a n). Arreglo ya ordenado.
- Duplicados: la condición $A[i] > \text{key}$ (estricta) garantiza *estabilidad*: elementos iguales no se reordenan entre sí.
- Ya ordenado: mejor caso $\Theta(n)$.
- Orden inverso: peor caso $\Theta(n^2)$.

Variantes del Invariante

- **Más fuerte:** $I'(j)$: “ $A[1..j - 1]$ ordenado + $A[j..n]$ sin modificar desde el original + permutación global”. Esta versión explícitamente rastrea la parte no procesada, útil para probar que no se pierden elementos.
- **Más débil:** $I''(j)$: “ $A[1..j - 1]$ está ordenado”. **Insuficiente** para correctitud completa: no garantiza que los elementos sean los mismos (podrían haberse perdido o duplicado).
- **0-based:** Si se usa indexación 0-based, $I(j)$: “ $A[0..j - 1]$ ordenado y permutación de $A_{\text{orig}}[0..j - 1]$ ” para $j = 1..n$.

Tips para Presentación

- **Traza recomendada:** $A = \langle 5, 2, 4, 6, 1, 3 \rangle$. Mostrar estado tras cada j : $j = 2 \rightarrow \langle 2, 5, 4, 6, 1, 3 \rangle$, $j = 3 \rightarrow \langle 2, 4, 5, 6, 1, 3 \rangle$, etc.
- **Visual:** Dibujar el subarreglo ordenado (sombreado) creciendo a la derecha. Flechas mostrando el “shift” de elementos.
- **Preguntas frecuentes:** “¿Por qué j empieza en 2?” (subarreglo de 1 elem ya ordenado). “¿Qué pasa con 0-based?” (cambia $j = 1$ y $i \geq 0$). “¿Es estable?” (sí, por $>$ estricto).
- **Demostración en vivo:** Usar array de 5–6 elementos, marcar key y mostrar desplazamientos.

2. Kadane's Algorithm — Máximo Subarreglo

Pseudocódigo

```

1  procedure KADANE( $A$ )
2       $max\_ending\_here = A[1]$ 
3       $max\_so\_far = A[1]$ 
4      for  $i = 2$  to  $A.length$ 
5           $max\_ending\_here = \max(A[i], max\_ending\_here + A[i])$ 
6           $max\_so\_far = \max(max\_so\_far, max\_ending\_here)$ 
7      return  $max\_so\_far$ 

```

Invariante de Loop

Al inicio de la iteración i (después de procesar $A[1..i-1]$, para $2 \leq i \leq n+1$):

$$max_ending_here = \max \left\{ \sum_{k=t}^{i-1} A[k] \mid 1 \leq t \leq i-1 \right\}$$

$$max_so_far = \max \left\{ \sum_{k=l}^r A[k] \mid 1 \leq l \leq r \leq i-1 \right\}$$

En palabras: max_ending_here es la suma máxima de un subarreglo que **termina** en $i-1$; max_so_far es la suma máxima de **cualquier** subarreglo contenido en $A[1..i-1]$.

Prueba de las Tres Propiedades

Inicialización ($i = 2$)

Antes de la primera iteración, solo se ha procesado $A[1]$.

- Subarreglos terminando en 1: solo $\{A[1]\}$, suma = $A[1]$. $max_ending_here = A[1]$. ✓
- Subarreglos en $A[1..1]$: solo $\{A[1]\}$, suma = $A[1]$. $max_so_far = A[1]$. ✓

Mantenimiento

HI: Invariante cierto para i (procesado $A[1..i-1]$). Procesamos $A[i]$.

- **Nuevo max_ending_here :** Un subarreglo terminando en i o es solo $\{A[i]\}$ (empezar nuevo) o extiende el óptimo terminando en $i-1$ (sumar $A[i]$ a max_ending_here anterior). La asignación $max_ending_here = \max(A[i], max_ending_here + A[i])$ captura exactamente esta disyuntiva.
- **Nuevo max_so_far :** El máximo subarreglo en $A[1..i]$ es o el anterior máximo (en $A[1..i-1]$) o el nuevo máximo terminando en i . La asignación $max_so_far = \max(max_so_far, max_ending_here)$ lo captura.

Por tanto el invariante se mantiene para $i+1$.

Terminación

Loop termina cuando $i = n + 1$. Entonces $max_so_far = \max\{\sum_{k=l}^r A[k] \mid 1 \leq l \leq r \leq n\}$, que es la definición de máximo subarreglo. Postcondición cumplida.

Complejidad

- **Temporal:** $\Theta(n)$ — un solo pase, $O(1)$ trabajo por elemento.
- **Espacial:** $\Theta(1)$ — solo 4 variables escalares.

Edge Cases

- **Todos negativos:** Retorna el máximo (el menos negativo, subarreglo de longitud 1). El invariante maneja esto porque $max(A[i], max_ending_here + A[i])$ elige $A[i]$ cuando extender empeora la suma.
- **Un elemento ($n = 1$):** Loop no ejecuta, retorna $A[1]$. Correcto.
- **Arreglo vacío:** Requiere guarda previa (retornar $-\infty$ o error).
- **Múltiples máximos:** Retorna la suma, no los índices (ver variante más fuerte).

Variantes del Invariante

- **Más fuerte (con índices):** Guardar también $(start_here, end_here)$ para max_ending_here y $(best_l, best_r)$ para max_so_far . Permite reconstruir el subarreglo óptimo.
- **Más débil:** Solo mantener max_so_far correcto. Pierde capacidad de reconstrucción y la intuición de “extend vs restart”.
- **Variante divide-y-vencerás (CLRS 4.1):** Invariante distinto: máximo está en izquierda, derecha, o cruza el medio. Complejidad $\Theta(n \log n)$.

Tips para Presentación

- **Traza recomendada:** $A = \langle -2, 1, -3, 4, -1, 2, 1, -5, 4 \rangle$. Máximo = 6 (subarreglo $[4, -1, 2, 1]$).
- **Tabla paso a paso:**

i	max_ending_here	max_so_far
1	-2	-2
2	1	1
3	-2	1
4	4	4
5	3	4
6	5	5
7	6	6
8	1	6
9	5	6

- **Intuición clave:** “En cada posición, ¿vale la pena extender el subarreglo actual o empezar uno nuevo?”.
- **Gráfico:** Plot de max_ending_here vs max_so_far a lo largo del array.

3. Sliding Window — Suma Máxima Ventana Fija k

Pseudocódigo

```

1  procedure MAX-SLIDING-WINDOW-SUM( $A, k$ )           ▷ Requiere  $1 \leq k \leq A.length$ 
2       $window\_sum = \sum_{i=1}^k A[i]$ 
3       $best = window\_sum$ 
4      for  $i = k + 1$  to  $A.length$ 
5           $window\_sum = window\_sum + A[i] - A[i - k]$ 
6           $best = \text{máx}(best, window\_sum)$ 
7      return  $best$ 

```

Invariante de Loop

Al inicio de la iteración i (para $k + 1 \leq i \leq n + 1$), después de la actualización de la ventana:

$$window_sum = \sum_{t=i-k}^{i-1} A[t]$$

$$best = \text{máx} \left\{ \sum_{t=j-k+1}^j A[t] \mid k \leq j \leq i-1 \right\}$$

Es decir: $window_sum$ es la suma de la ventana actual de tamaño k que termina en $i - 1$; $best$ es la máxima suma de ventana vista hasta $i - 1$.

Prueba de las Tres Propiedades

Inicialización ($i = k + 1$)

Antes del loop, se computa $window_sum = \sum_{t=1}^k A[t]$, que es la suma de la primera ventana ($A[1..k]$). Como $i = k + 1$, la ventana actual termina en $i - 1 = k$, coincidiendo con la definición. $best = window_sum$ es el máximo sobre la única ventana vista. Invariante se cumple para $i = k + 1$.

Mantenimiento

HI: Invariante cierto para i (ventana actual $A[i - k..i - 1]$, $best$ correcto hasta $i - 1$). Al avanzar a $i + 1$, la nueva ventana es $A[i - k + 1..i]$. Pierde $A[i - k]$ y gana $A[i]$. La actualización

$$window_sum \leftarrow window_sum + A[i] - A[i - k]$$

refleja exactamente este cambio. $best \leftarrow \text{máx}(best, window_sum)$ incorpora la nueva ventana al máximo histórico. Invariante se mantiene para $i + 1$.

Terminación

Loop termina cuando $i = n + 1$. Se han evaluado todas las $n - k + 1$ ventanas posibles (índices finales $k, k + 1, \dots, n$). $best$ es el máximo global sobre todas las ventanas de tamaño k .

Complejidad

- **Temporal:** $\Theta(n)$. Suma inicial $O(k)$, loop $n - k$ iteraciones con $O(1)$ cada una. Total $O(k + n - k) = O(n)$.
- **Espacial:** $\Theta(1)$ — tres variables escalares.

Comparación: fuerza bruta recalcula suma cada ventana $\rightarrow \Theta(nk)$. Sliding window ahorra factor k .

Edge Cases

- $k = n$: una sola ventana (el arreglo completo). Loop no ejecuta (i va de $n + 1$ a n). Retorna suma total. Correcto.
- $k = 1$: ventana de tamaño 1. $window_sum = A[i]$, $best = \max(A)$. Equivale a encontrar máximo elemento.
- $k > n$: precondition violada. Manejar con error o retornar suma total si se relaja.
- Valores negativos: funciona sin cambios. $best$ puede ser negativo.

Variantes del Invariante

- **Más fuerte (con índice):** Guardar $best_start = \text{índice de inicio de la mejor ventana}$. Permite reportar posición.
- **Ventana variable (two pointers):** Tamaño k no fijo, sino condición sobre suma (ej. $suma \leq S$). Invariante más complejo: mantener ventana válida y máxima.
- **Mínimo en lugar de máximo:** Cambiar $\max$ por $\min$, invariante análogo.

Tips para Presentación

- **Traza recomendada:** $A = \langle 1, 3, -1, -3, 5, 3, 6, 7 \rangle$, $k = 3$.

Ventana	Elementos	Suma
1	[1, 3, -1]	3
2	[3, -1, -3]	-1
3	[-1, -3, 5]	1
4	[-3, 5, 3]	5
5	[5, 3, 6]	14
6	[3, 6, 7]	16 $\leftarrow$ max

- **Visual:** Rectángulo deslizándose sobre array, flechas “sale $A[i - k]$ ”, “entra $A[i]$ ”.
- **Comparación:** Mostrar pseudo-fuerza bruta ($O(nk)$) vs sliding window ($O(n)$).

4. Prefix Sum (Suma de Prefijos)

Pseudocódigo

```

1  procedure PREFIX-SUM( $A$ )
2       $P[0] = 0$ 
3      for  $i = 1$  to  $A.length$ 
4           $P[i] = P[i - 1] + A[i]$ 
5      return  $P$ 

```

Range Query (Consulta de Rango)

```

1  procedure RANGE-SUM( $P, l, r$ )
2      return  $P[r] - P[l - 1]$ 

```

▷ Requiere $1 \leq l \leq r \leq n$

Invariante de Loop

Al inicio de la iteración i (para $1 \leq i \leq n + 1$), después de procesar $A[1..i - 1]$:

$$P[i - 1] = \sum_{k=1}^{i-1} A[k] \quad (\text{suma de prefijos hasta } i - 1).$$

Para todo $j \leq i - 1$: $P[j] = \sum_{k=1}^j A[k]$.

Prueba de las Tres Propiedades

Inicialización ($i = 1$)

Antes de la primera iteración, $i = 1$. $P[0] = 0$ por definición. La suma vacía $\sum_{k=1}^0 A[k] = 0$. El invariante se cumple para $P[0]$.

Mantenimiento

HI: $P[i - 1] = \sum_{k=1}^{i-1} A[k]$. Al procesar $A[i]$, asignamos $P[i] = P[i - 1] + A[i] = \sum_{k=1}^{i-1} A[k] + A[i] = \sum_{k=1}^i A[k]$. El invariante se mantiene para $i + 1$.

Terminación

Loop termina cuando $i = n + 1$. Entonces $P[n] = \sum_{k=1}^n A[k]$ y para todo $j \leq n$, $P[j] = \sum_{k=1}^j A[k]$. El array P contiene todas las sumas de prefijos. La consulta $P[r] - P[l - 1]$ retorna $\sum_{k=l}^r A[k]$ en $O(1)$.

Complejidad

- **Construcción:** $\Theta(n)$ tiempo, $\Theta(n)$ espacio (array P de tamaño $n + 1$).
- **Consulta Range-Sum:** $\Theta(1)$ tiempo, $\Theta(1)$ espacio extra.

Edge Cases

- $n = 0$: $P = [0]$, consulta inválida (manejar error).
- $l = r$: suma de un solo elemento $A[l] = P[l] - P[l - 1]$.
- $l = 1$: suma de prefijo $P[r]$ directamente.
- Valores negativos: funciona sin cambios.
- Overflow: en lenguajes con enteros fijos, usar tipo mayor o módulo.

Variantes del Invariante

- **Más fuerte (2D)**: Matriz de prefijos $P[i][j] = \sum_{x \leq i, y \leq j} A[x][y]$. Consulta submatriz en $O(1)$.
- **Más fuerte (producto)**: Prefix product para consultas de producto de rango (con división o logs).
- **Más débil**: Solo $P[n]$ (suma total). Pierde capacidad de consultas de rango arbitrario.
- **Diferencias finitas**: Inverso de prefix sum: $A[i] = P[i] - P[i - 1]$.

Tips para Presentación

- **Traza recomendada**: $A = \langle 3, 1, 4, 1, 5, 9 \rangle \rightarrow P = \langle 0, 3, 4, 8, 9, 14, 23 \rangle$.
- **Consulta ejemplo**: $l = 2, r = 5 \rightarrow P[5] - P[1] = 14 - 3 = 11 = 1 + 4 + 1 + 5$.
- **Visual**: Arreglo A arriba, array P abajo, flechas mostrando acumulación.
- **Intuición clave**: “Precomputar para responder rápido”. Trade-off: $O(n)$ preproceso $\rightarrow O(1)$ por consulta.
- **Preguntas frecuentes**: “¿Y si el array se actualiza?” (Fenwick tree / Segment tree). “¿Funciona en 2D?” (sí, integral image).

Resumen de Rúbrica para Corrección

Categoría	Pts	Qué Buscar
Correctness (30)	30	Invariante matemáticamente preciso; 3 propiedades probadas paso a paso; complejidad con justificación; edge cases mencionados
Clarity & Structure (25)	25	Pseudocódigo CLRS impecable; secciones ordenadas; notación consistente; trazas claras
Depth of Analysis (25)	25	Variantes de invariante discutidas; pre/postcondiciones; comparación con otros algoritmos; limitaciones honestas
Presentation (20)	20	Slides visuales; demo en vivo fluida; respuestas seguras; tiempo OK; pareja equilibrada

Total: 100 pts